

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Output:

```
Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape: (10000, 28, 28)
Testing labels shape: (10000,)
Number of classes: 10
```

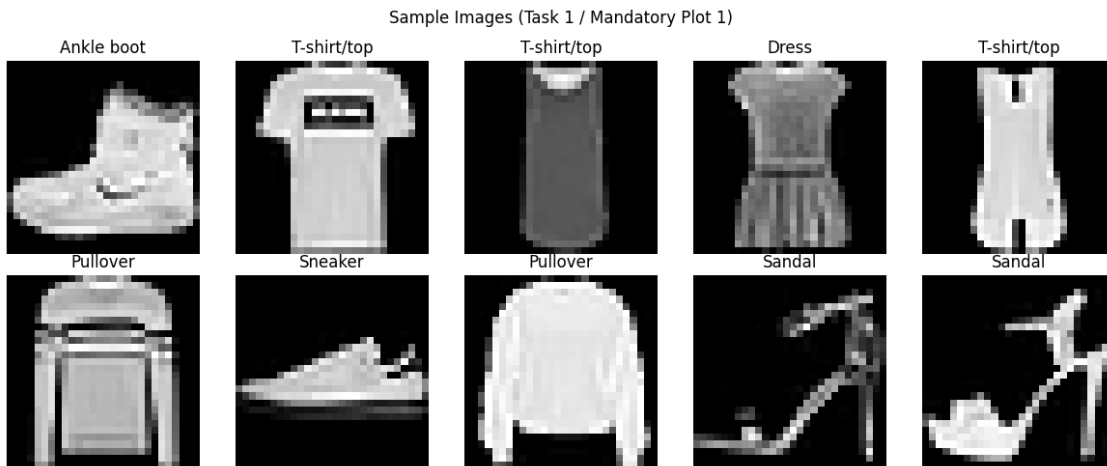


Figure 1: Sample Images from Fashion-MNIST (Mandatory Plot 1)

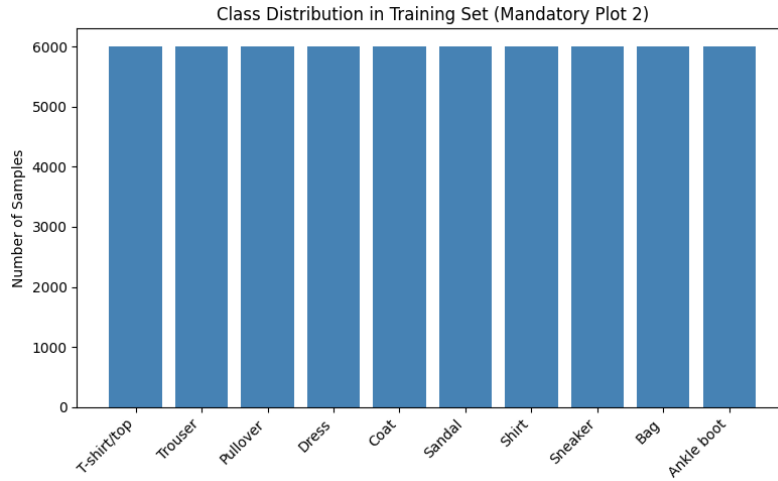


Figure 2: Class Distribution in the Training Set (Mandatory Plot 2)

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Output:

Before preprocessing:

x_train shape: (60000, 28, 28) | dtype: uint8
x_test shape: (10000, 28, 28)

After preprocessing:

x_train_flat shape: (60000, 784) | dtype: float32
x_test_flat shape: (10000, 784)
y_train_onehot shape: (60000, 10)
y_test_onehot shape: (10000, 10)
Pixel value range: 0.0 - 1.0

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params	109,386 (427.29 KB)
Trainable params	109,386 (427.29 KB)
Non-trainable params	0 (0.00 B)

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Output:

```
Epoch 1/20
1688/1688 22s 12ms/step - accuracy: 0.8203 - loss: 0.5028 - val_accuracy: 0.8532 - val_loss: 0.4123
Epoch 2/20
1688/1688 12s 7ms/step - accuracy: 0.8646 - loss: 0.3731 - val_accuracy: 0.8607 - val_loss: 0.3639
Epoch 3/20
1688/1688 8s 5ms/step - accuracy: 0.8766 - loss: 0.3359 - val_accuracy: 0.8623 - val_loss: 0.3605
Epoch 4/20
1688/1688 7s 4ms/step - accuracy: 0.8851 - loss: 0.3103 - val_accuracy: 0.8770 - val_loss: 0.3342
Epoch 5/20
1688/1688 8s 5ms/step - accuracy: 0.8928 - loss: 0.2895 - val_accuracy: 0.8692 - val_loss: 0.3586
Epoch 6/20
1688/1688 8s 5ms/step - accuracy: 0.8974 - loss: 0.2752 - val_accuracy: 0.8743 - val_loss: 0.3473
Epoch 7/20
1688/1688 7s 4ms/step - accuracy: 0.9019 - loss: 0.2615 - val_accuracy: 0.8703 - val_loss: 0.3776
Epoch 8/20
1688/1688 9s 5ms/step - accuracy: 0.9064 - loss: 0.2490 - val_accuracy: 0.8753 - val_loss: 0.3684
Epoch 9/20
1688/1688 7s 4ms/step - accuracy: 0.9101 - loss: 0.2383 - val_accuracy: 0.8787 - val_loss: 0.3535
Epoch 10/20
1688/1688 8s 5ms/step - accuracy: 0.9143 - loss: 0.2282 - val_accuracy: 0.8803 - val_loss: 0.3698
Epoch 11/20
1688/1688 7s 4ms/step - accuracy: 0.9166 - loss: 0.2229 - val_accuracy: 0.8772 - val_loss: 0.3812
Epoch 12/20
1688/1688 9s 5ms/step - accuracy: 0.9190 - loss: 0.2131 - val_accuracy: 0.8808 - val_loss: 0.3795
Epoch 13/20
1688/1688 8s 5ms/step - accuracy: 0.9221 - loss: 0.2050 - val_accuracy: 0.8880 - val_loss: 0.3725
Epoch 14/20
1688/1688 7s 4ms/step - accuracy: 0.9249 - loss: 0.1992 - val_accuracy: 0.8835 - val_loss: 0.3908
Epoch 15/20
1688/1688 8s 5ms/step - accuracy: 0.9272 - loss: 0.1914 - val_accuracy: 0.8838 - val_loss: 0.3950
Epoch 16/20
1688/1688 7s 4ms/step - accuracy: 0.9305 - loss: 0.1852 - val_accuracy: 0.8798 - val_loss: 0.4239
Epoch 17/20
1688/1688 8s 5ms/step - accuracy: 0.9314 - loss: 0.1810 - val_accuracy: 0.8828 - val_loss: 0.4206
Epoch 18/20
1688/1688 11s 5ms/step - accuracy: 0.9325 - loss: 0.1758 - val_accuracy: 0.8790 - val_loss: 0.4290
Epoch 19/20
1688/1688 7s 4ms/step - accuracy: 0.9364 - loss: 0.1686 - val_accuracy: 0.8875 - val_loss: 0.4013
Epoch 20/20
1688/1688 8s 5ms/step - accuracy: 0.9360 - loss: 0.1668 - val_accuracy: 0.8817 - val_loss: 0.4343
```

Baseline training time: 175.97 seconds

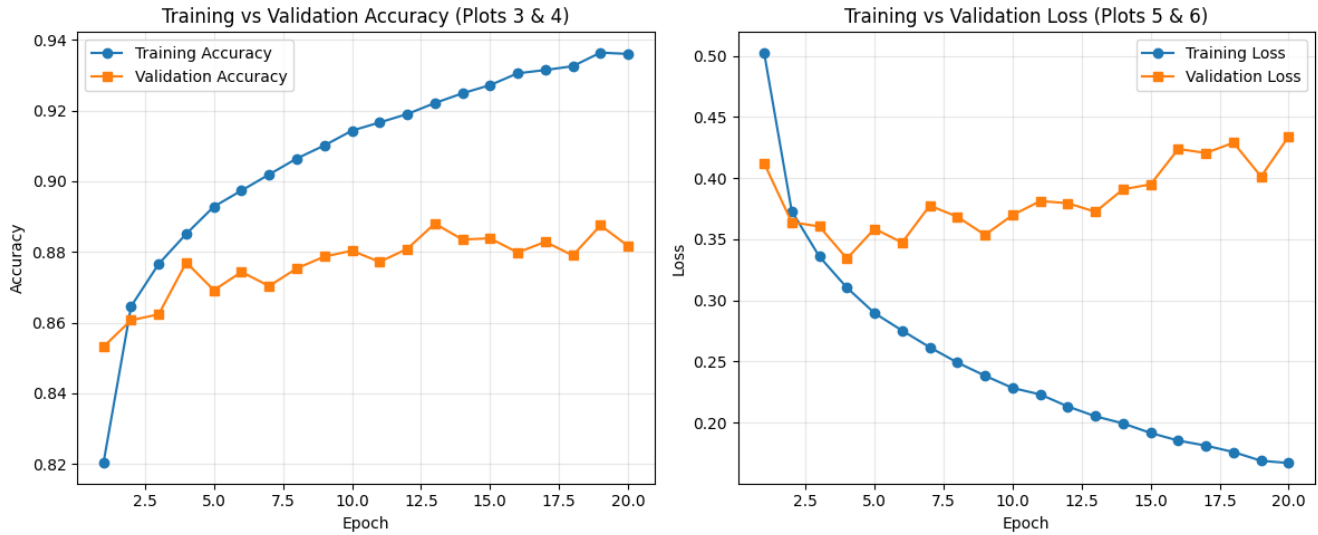


Figure 3: Training vs Validation Accuracy and Loss (Mandatory Plots 3–6)

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Output:

Baseline Test Accuracy: 0.8796
 Baseline Precision: 0.8797
 Baseline Recall: 0.8796
 Baseline F1-score: 0.8786

Classification Report:

	precision	recall	f1-score	support
T-shirt/top	0.81	0.86	0.83	1000
Trouser	1.00	0.94	0.97	1000
Pullover	0.78	0.80	0.79	1000
Dress	0.84	0.93	0.88	1000
Coat	0.78	0.81	0.79	1000
Sandal	0.96	0.97	0.96	1000
Shirt	0.76	0.63	0.69	1000
Sneaker	0.94	0.96	0.95	1000
Bag	0.96	0.97	0.96	1000
Ankle boot	0.97	0.94	0.96	1000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000

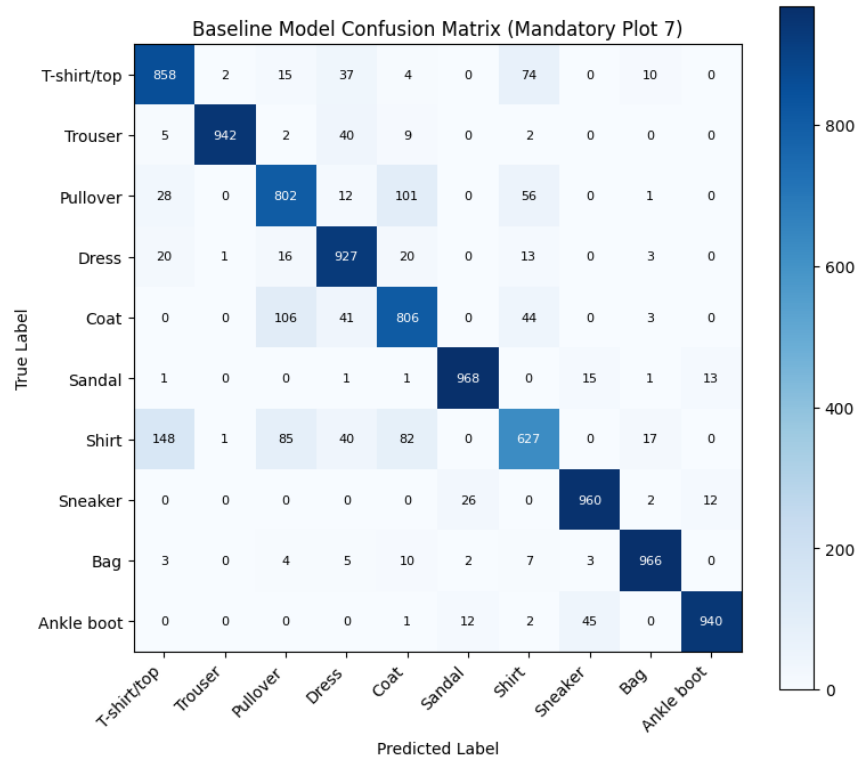


Figure 4: Baseline Model Confusion Matrix (Mandatory Plot 7)

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.

Output: Baseline model reused from Task 3 (see model summary above).

2. Define the hyperparameter search space.

Output:

Search space defined with 11664 possible combinations.

3. Perform Grid Search or Randomized Search using 5-fold cross-validation.

Output:

```
[1/20] mean CV accuracy: 0.8335 | params: {'optimizer_name': 'adam', 'learning_rate': 0.001,
      'hidden_neurons': 256, 'hidden_layers': 1, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 128, 'activation': 'tanh'}
[2/20] mean CV accuracy: 0.5897 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.01,
      'hidden_neurons': 256, 'hidden_layers': 3, 'epochs': 20, 'dropout_rate': 0.5,
      'batch_size': 16, 'activation': 'relu'}
[3/20] mean CV accuracy: 0.8365 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.001,
      'hidden_neurons': 128, 'hidden_layers': 3, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 32, 'activation': 'tanh'}
[4/20] mean CV accuracy: 0.8280 | params: {'optimizer_name': 'adam', 'learning_rate': 0.001,
      'hidden_neurons': 32, 'hidden_layers': 1, 'epochs': 10, 'dropout_rate': 0.2,
      'batch_size': 32, 'activation': 'tanh'}
[5/20] mean CV accuracy: 0.8313 | params: {'optimizer_name': 'adam', 'learning_rate': 0.001,
      'hidden_neurons': 64, 'hidden_layers': 2, 'epochs': 30, 'dropout_rate': 0.2,
      'batch_size': 128, 'activation': 'sigmoid'}
[6/20] mean CV accuracy: 0.3527 | params: {'optimizer_name': 'adam', 'learning_rate': 0.1,
      'hidden_neurons': 64, 'hidden_layers': 1, 'epochs': 30, 'dropout_rate': 0.5,
      'batch_size': 32, 'activation': 'tanh'}
[7/20] mean CV accuracy: 0.6302 | params: {'optimizer_name': 'adam', 'learning_rate': 0.1,
      'hidden_neurons': 32, 'hidden_layers': 1, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 64, 'activation': 'tanh'}
[8/20] mean CV accuracy: 0.8418 | params: {'optimizer_name': 'adam', 'learning_rate': 0.001,
      'hidden_neurons': 256, 'hidden_layers': 1, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 16, 'activation': 'relu'}
[9/20] mean CV accuracy: 0.8277 | params: {'optimizer_name': 'adam', 'learning_rate': 0.001,
      'hidden_neurons': 256, 'hidden_layers': 3, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 16, 'activation': 'tanh'}
[10/20] mean CV accuracy: 0.8270 | params: {'optimizer_name': 'adam', 'learning_rate': 0.001,
      'hidden_neurons': 256, 'hidden_layers': 2, 'epochs': 10, 'dropout_rate': 0.5,
      'batch_size': 32, 'activation': 'tanh'}
[11/20] mean CV accuracy: 0.4873 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.001,
      'hidden_neurons': 32, 'hidden_layers': 3, 'epochs': 30, 'dropout_rate': 0.5,
      'batch_size': 128, 'activation': 'sigmoid'}
[12/20] mean CV accuracy: 0.8163 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.01,
      'hidden_neurons': 128, 'hidden_layers': 1, 'epochs': 10, 'dropout_rate': 0.5,
      'batch_size': 128, 'activation': 'sigmoid'}
[13/20] mean CV accuracy: 0.7770 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.01,
      'hidden_neurons': 128, 'hidden_layers': 3, 'epochs': 10, 'dropout_rate': 0.2,
      'batch_size': 128, 'activation': 'sigmoid'}
[14/20] mean CV accuracy: 0.6750 | params: {'optimizer_name': 'sgd', 'learning_rate': 0.001,
      'hidden_neurons': 32, 'hidden_layers': 1, 'epochs': 30, 'dropout_rate': 0.2,
      'batch_size': 16, 'activation': 'sigmoid'}
[15/20] mean CV accuracy: 0.1062 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.1,
      'hidden_neurons': 256, 'hidden_layers': 2, 'epochs': 10, 'dropout_rate': 0.5,
      'batch_size': 32, 'activation': 'relu'}
[16/20] mean CV accuracy: 0.7368 | params: {'optimizer_name': 'adam', 'learning_rate': 0.01,
      'hidden_neurons': 64, 'hidden_layers': 1, 'epochs': 20, 'dropout_rate': 0.5,
      'batch_size': 16, 'activation': 'relu'}
[17/20] mean CV accuracy: 0.8117 | params: {'optimizer_name': 'rmsprop', 'learning_rate': 0.001,
      'hidden_neurons': 256, 'hidden_layers': 3, 'epochs': 20, 'dropout_rate': 0.5,
      'batch_size': 64, 'activation': 'sigmoid'}
[18/20] mean CV accuracy: 0.4742 | params: {'optimizer_name': 'adam', 'learning_rate': 0.1,
      'hidden_neurons': 32, 'hidden_layers': 2, 'epochs': 20, 'dropout_rate': 0.0,
      'batch_size': 128, 'activation': 'tanh'}
[19/20] mean CV accuracy: 0.7878 | params: {'optimizer_name': 'sgd', 'learning_rate': 0.01,
      'hidden_neurons': 128, 'hidden_layers': 2, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 64, 'activation': 'relu'}
[20/20] mean CV accuracy: 0.4440 | params: {'optimizer_name': 'adam', 'learning_rate': 0.1,
      'hidden_neurons': 128, 'hidden_layers': 1, 'epochs': 20, 'dropout_rate': 0.2,
      'batch_size': 32, 'activation': 'tanh'}
```

Hyperparameter search completed in 1435.83 seconds

4. Record the best hyperparameter combination.

Output:

Best hyperparameters found:

```
optimizer_name: adam
learning_rate: 0.001
hidden_neurons: 256
hidden_layers: 1
epochs: 20
dropout_rate: 0.2
batch_size: 16
activation: relu
```

Best cross-validation accuracy: 0.8418

5. Retrain the model using the optimized hyperparameters.

Output:

```
Epoch 1/20
3375/3375 19s 5ms/step - accuracy: 0.8160 - loss: 0.5135 - val_accuracy: 0.8540 - val_loss: 0.3936
Epoch 2/20
3375/3375 17s 5ms/step - accuracy: 0.8556 - loss: 0.3964 - val_accuracy: 0.8615 - val_loss: 0.3741
Epoch 3/20
3375/3375 17s 5ms/step - accuracy: 0.8662 - loss: 0.3633 - val_accuracy: 0.8700 - val_loss: 0.3560
Epoch 4/20
3375/3375 21s 5ms/step - accuracy: 0.8738 - loss: 0.3411 - val_accuracy: 0.8713 - val_loss: 0.3551
Epoch 5/20
3375/3375 17s 5ms/step - accuracy: 0.8794 - loss: 0.3245 - val_accuracy: 0.8765 - val_loss: 0.3463
Epoch 6/20
3375/3375 17s 5ms/step - accuracy: 0.8847 - loss: 0.3113 - val_accuracy: 0.8767 - val_loss: 0.3459
Epoch 7/20
3375/3375 21s 5ms/step - accuracy: 0.8888 - loss: 0.2992 - val_accuracy: 0.8758 - val_loss: 0.3456
Epoch 8/20
3375/3375 17s 5ms/step - accuracy: 0.8913 - loss: 0.2907 - val_accuracy: 0.8800 - val_loss: 0.3449
Epoch 9/20
3375/3375 17s 5ms/step - accuracy: 0.8946 - loss: 0.2807 - val_accuracy: 0.8797 - val_loss: 0.3444
Epoch 10/20
3375/3375 18s 5ms/step - accuracy: 0.8975 - loss: 0.2727 - val_accuracy: 0.8832 - val_loss: 0.3507
Epoch 11/20
3375/3375 17s 5ms/step - accuracy: 0.8992 - loss: 0.2681 - val_accuracy: 0.8817 - val_loss: 0.3412
Epoch 12/20
3375/3375 16s 5ms/step - accuracy: 0.9010 - loss: 0.2610 - val_accuracy: 0.8800 - val_loss: 0.3631
Epoch 13/20
3375/3375 17s 5ms/step - accuracy: 0.9051 - loss: 0.2546 - val_accuracy: 0.8878 - val_loss: 0.3420
Epoch 14/20
3375/3375 21s 5ms/step - accuracy: 0.9058 - loss: 0.2508 - val_accuracy: 0.8865 - val_loss: 0.3664
Epoch 15/20
3375/3375 16s 5ms/step - accuracy: 0.9070 - loss: 0.2451 - val_accuracy: 0.8835 - val_loss: 0.3509
Epoch 16/20
3375/3375 17s 5ms/step - accuracy: 0.9104 - loss: 0.2378 - val_accuracy: 0.8855 - val_loss: 0.3628
Epoch 17/20
3375/3375 20s 5ms/step - accuracy: 0.9108 - loss: 0.2365 - val_accuracy: 0.8787 - val_loss: 0.3757
Epoch 18/20
3375/3375 17s 5ms/step - accuracy: 0.9135 - loss: 0.2303 - val_accuracy: 0.8875 - val_loss: 0.3668
Epoch 19/20
3375/3375 17s 5ms/step - accuracy: 0.9148 - loss: 0.2264 - val_accuracy: 0.8840 - val_loss: 0.3821
Epoch 20/20
3375/3375 18s 5ms/step - accuracy: 0.9143 - loss: 0.2235 - val_accuracy: 0.8872 - val_loss: 0.3878
```

Optimized model training time: 353.73 seconds

6. Evaluate the optimized model on the testing dataset.

Output:

Optimized Test Accuracy: 0.8861

Optimized Precision: 0.8877
 Optimized Recall: 0.8861
 Optimized F1-score: 0.8835

Classification Report (Optimized Model):

	precision	recall	f1-score	support
T-shirt/top	0.78	0.89	0.83	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.78	0.83	0.80	1000
Dress	0.84	0.93	0.88	1000
Coat	0.79	0.83	0.81	1000
Sandal	0.98	0.97	0.97	1000
Shirt	0.82	0.55	0.66	1000
Sneaker	0.95	0.95	0.95	1000
Bag	0.99	0.97	0.98	1000
Ankle boot	0.96	0.97	0.96	1000
accuracy			0.89	10000
macro avg	0.89	0.89	0.88	10000
weighted avg	0.89	0.89	0.88	10000

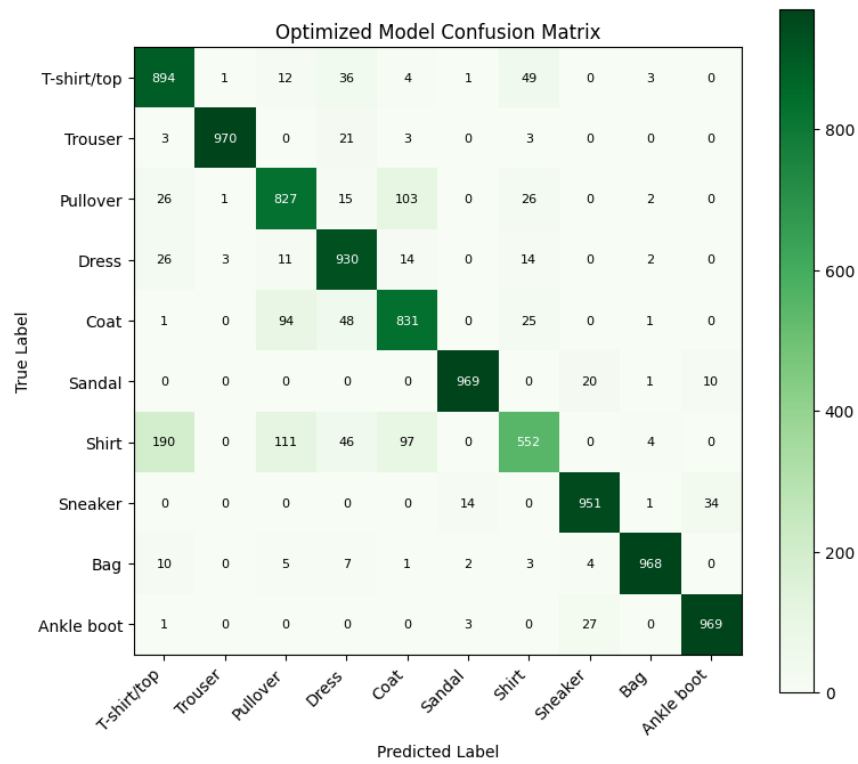


Figure 5: Optimized Model Confusion Matrix

7. Compare the optimized model with the baseline model.

Output:

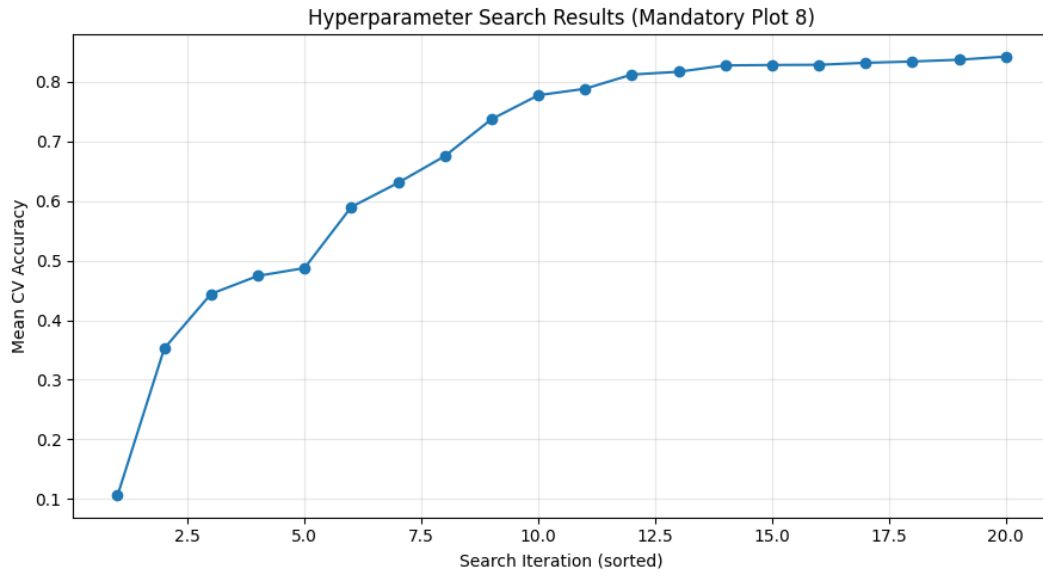


Figure 6: Hyperparameter Search Results (Mandatory Plot 8)

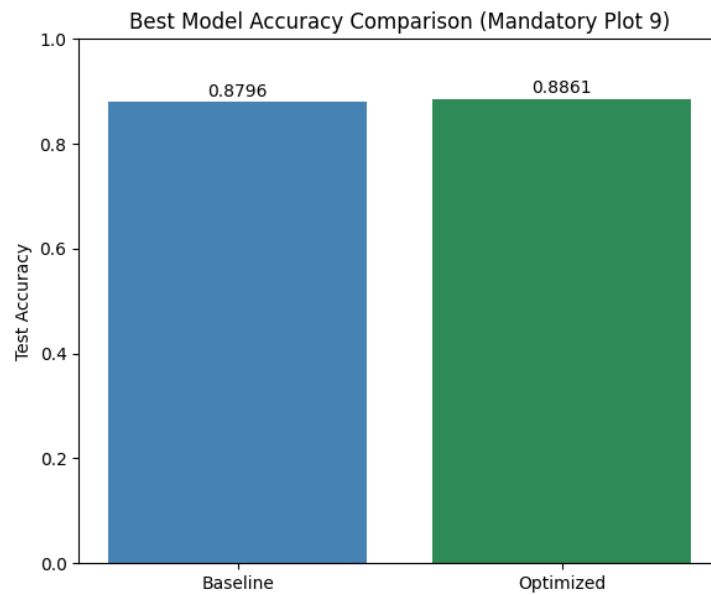


Figure 7: Best Model Accuracy Comparison (Mandatory Plot 9)

Best Hyperparameters

```

-----
Hidden Layers      : 1
Hidden Neurons    : 256
Learning Rate     : 0.001
Batch Size        : 16
Optimizer         : adam
Activation Function: relu
Epochs           : 20
Dropout           : 0.2
Cross-validation Accuracy : 0.8418
Testing Accuracy  : 0.8861
  
```

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8796	0.8861
Precision	0.8797	0.8877
Recall	0.8796	0.8861
F1-score	0.8786	0.8835
Training Time(s)	175.97	353.73

8. Mandatory Plots

Generate the following plots.

1. Sample Images
 2. Class Distribution
 3. Training Accuracy vs Epoch
 4. Validation Accuracy vs Epoch
 5. Training Loss vs Epoch
 6. Validation Loss vs Epoch
 7. Confusion Matrix
 8. Hyperparameter Search Results
 9. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	ReLU
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8418
Testing Accuracy	0.8861

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8796	0.8861
Precision	0.8797	0.8877
Recall	0.8796	0.8861
F1-score	0.8786	0.8835
Training Time	175.97 s	353.73 s

10. Discussion

Answer the following:

1. Which optimization method was used?
2. Which hyperparameters were selected?
3. Did optimization improve performance?
4. Which hyperparameter had the greatest impact?
5. Compare Grid Search and Randomized Search.
6. Recommend the best MLP configuration.

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

13. Additional Exercise: Perceptron Learning Algorithm for the XOR Gate (K4)

Problem Statement: Code the perceptron learning algorithm for the XOR gate. Show the weights after each update. Plot the decision boundary after each weight update. Analyze and identify the pattern that prevents the algorithm from converging.

13.1 Output: Weights After Each Update

```
Update 1 (Epoch 1, sample x0=[0 0], target=0, predicted=1)
  Weights = [0. 0.], Bias = -1
Update 2 (Epoch 1, sample x1=[0 1], target=1, predicted=0)
  Weights = [0. 1.], Bias = 0
Update 3 (Epoch 1, sample x3=[1 1], target=0, predicted=1)
  Weights = [-1. 0.], Bias = -1
Update 4 (Epoch 2, sample x1=[0 1], target=1, predicted=0)
  Weights = [-1. 1.], Bias = 0
Update 5 (Epoch 2, sample x2=[1 0], target=1, predicted=0)
  Weights = [0. 1.], Bias = 1
Update 6 (Epoch 2, sample x3=[1 1], target=0, predicted=1)
  Weights = [-1. 0.], Bias = 0
Update 7 (Epoch 3, sample x0=[0 0], target=0, predicted=1)
  Weights = [-1. 0.], Bias = -1
Update 8 (Epoch 3, sample x1=[0 1], target=1, predicted=0)
  Weights = [-1. 1.], Bias = 0
Update 9 (Epoch 3, sample x2=[1 0], target=1, predicted=0)
  Weights = [0. 1.], Bias = 1
Update 10 (Epoch 3, sample x3=[1 1], target=0, predicted=1)
  Weights = [-1. 0.], Bias = 0
Update 11 (Epoch 4, sample x0=[0 0], target=0, predicted=1)
  Weights = [-1. 0.], Bias = -1
Update 12 (Epoch 4, sample x1=[0 1], target=1, predicted=0)
  Weights = [-1. 1.], Bias = 0
Update 13 (Epoch 4, sample x2=[1 0], target=1, predicted=0)
  Weights = [0. 1.], Bias = 1
Update 14 (Epoch 4, sample x3=[1 1], target=0, predicted=1)
  Weights = [-1. 0.], Bias = 0
Update 15 (Epoch 5, sample x0=[0 0], target=0, predicted=1)
  Weights = [-1. 0.], Bias = -1
Update 16 (Epoch 5, sample x1=[0 1], target=1, predicted=0)
  Weights = [-1. 1.], Bias = 0
Update 17 (Epoch 5, sample x2=[1 0], target=1, predicted=0)
  Weights = [0. 1.], Bias = 1
Update 18 (Epoch 5, sample x3=[1 1], target=0, predicted=1)
  Weights = [-1. 0.], Bias = 0
```

Stopped after reaching max_epochs = 5 WITHOUT converging (errors persist every epoch).

Final Weights: [-1. 0.]

Final Bias: 0

Total weight updates performed: 18

13.2 Output: Decision Boundary After Each Weight Update

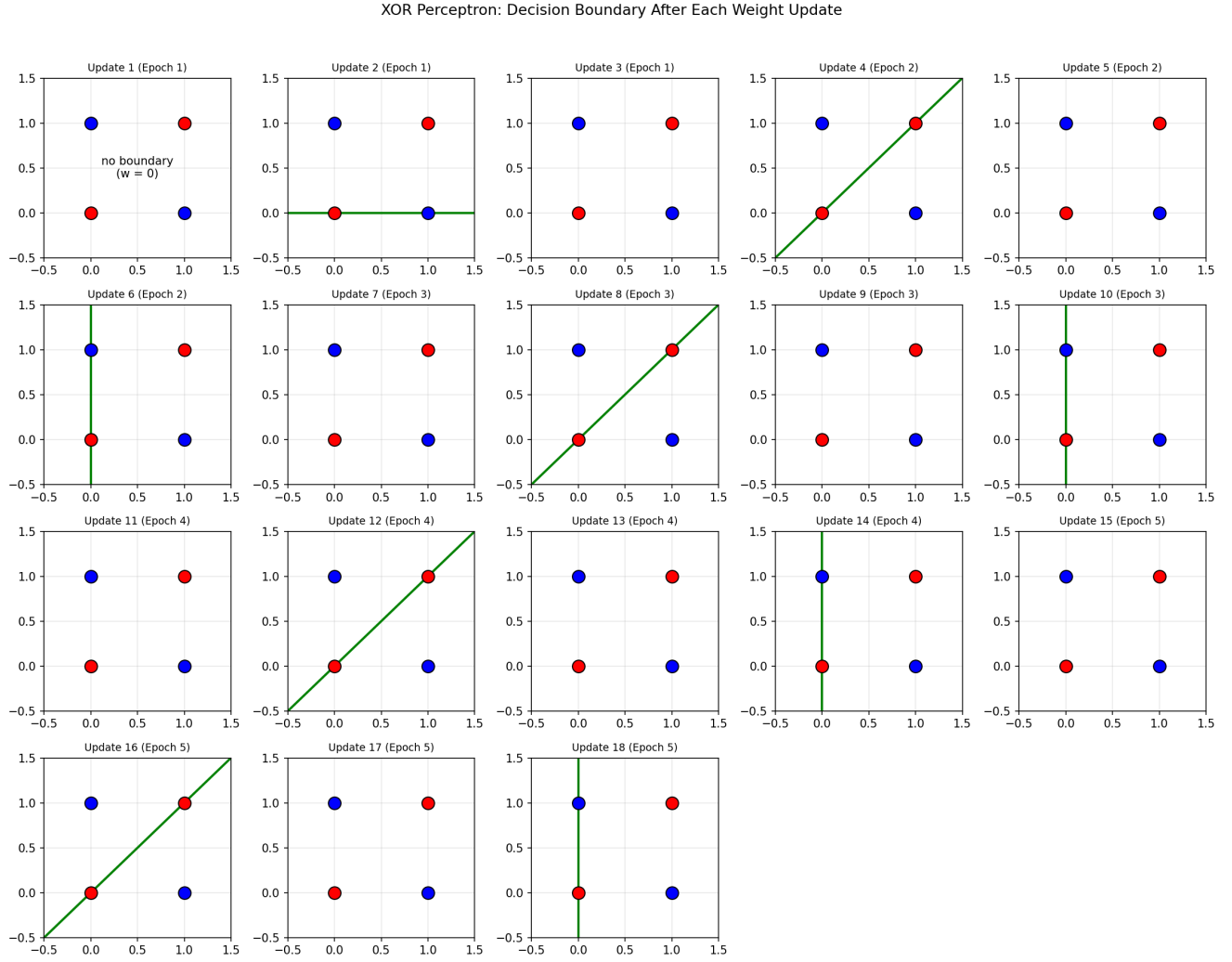


Figure 8: Decision boundary of the perceptron after every weight update while training on the XOR gate. Red points are class 0, blue points are class 1.

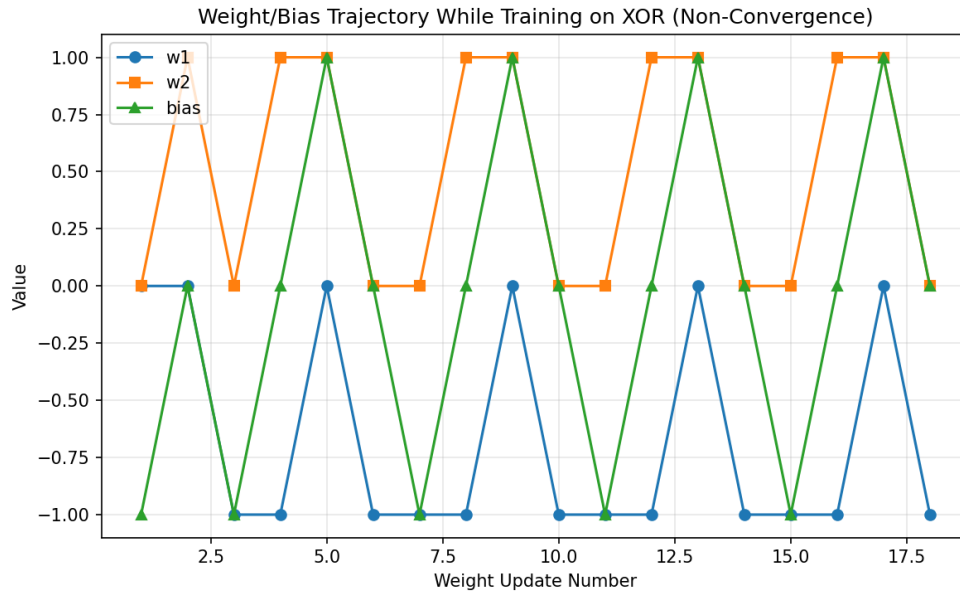


Figure 9: Trajectory of w_1 , w_2 , and bias across successive weight updates, showing the repeating cycle instead of convergence.

13.3 Analysis / Inference

- From Epoch 3 onward, the weights and bias repeat the exact same 4-update sequence every epoch: $(-1, 0, -1) \rightarrow (-1, 1, 0) \rightarrow (0, 1, 1) \rightarrow (-1, 0, 0)$, and then back to $(-1, 0, -1)$ again. The algorithm has entered a closed cycle rather than approaching a fixed set of weights.
- This cyclic behaviour occurs because the four XOR points *cannot* be separated by any single straight line – the two positive points $(0, 1)$ and $(1, 0)$ lie on opposite corners from the two negative points $(0, 0)$ and $(1, 1)$, which is precisely the non-linearly-separable configuration. No choice of w_1 , w_2 , b can classify all four points correctly at once.
- Every full pass through the dataset therefore always misclassifies at least one point, so the error-driven weight update never reaches an epoch with zero errors. Instead of converging, the weight vector oscillates indefinitely through the same states, as confirmed by the flat, repeating pattern in Figure 13 (weight trajectory) after update 6.
- This is a direct, practical demonstration of the Perceptron Convergence Theorem: the perceptron learning rule is guaranteed to converge in a finite number of steps *only if* the training data is linearly separable. XOR is the canonical counter-example, and solving it requires a multi-layer network with a hidden layer (as implemented separately using backpropagation), which can learn a non-linear decision boundary.